Pandas Library

Abdesselam Filali

2026-02-11

1 Overview

The pandas library provides high-performance, easy-to-use data structures and data analysis tools. The main data structure is the DataFrame, which you can think of as an in-memory 2D table (like a spreadsheet, with column names and row labels).

Many features available in Excel are available programmatically, such as creating pivot tables, computing columns based on other columns, plotting graphs, etc. You can also group rows by column value, or join tables much like in SQL. Pandas is also great at handling time series data.

Built on top of the Python programming language. pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, it is used for working with data sets.It has functions for analyzing, cleaning, exploring, and manipulating data.

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Pandas allows to analyze big data and make conclusions based on statistical theories.

Pandas can clean messy data sets and make them readable and relevant.

Relevant data is very important in data science.

`https://pandas.pydata.org/`

2 Why pandas?

Pandas is a powerful tool for data analysis and manipulation. It provides a wide range of data structures, such as Series and DataFrame, which are optimized for performance and ease of use. Pandas also provides a large number of functions for data manipulation, such as filtering, sorting, and aggregating data.

Pandas is also a great tool for data visualization. It provides a wide range of plotting functions, such as line plots, bar plots, and scatter plots, which can be used to visualize data in a variety of ways.

3 Background

It's possible that Python wouldn't have become the lingua franca of data science if it wasn't for pandas. The package's exponential growth on Stack Overflow means two things:

It's getting increasingly popular.

It can be frustrating to use sometimes (hence the high number of questions).

This tutorial contains a few peculiar things about pandas that hopefully make your life easier and code faster.

4 Prerequisites

If you are not familiar with NumPy, we recommend that you go through the NumPy tutorial now.

5 Installing

```
pip install pandas
```

```
pip install --upgrade pandas
```

6 Importing

```
import pandas as pd
```

7 Pandas datastructures

7.1 Pandas Series

Pandas Series –> 1 dimension data (a sequence)

A Pandas Series is a one-dimensional array with axis labels, also known as a single column of data. It is similar to a list, but with a few key differences:

- A Series can have a name.
- A Series can have a data type.
- A Series can have a index, which is a sequence of labels that can be used to access the data.

7.2 Pandas DataFrames

Pandas DataFrames –> 2 dimensions (columns as variables)

A Pandas DataFrame is a two-dimensional array with axis labels, also known as a table of data. It is similar to a spreadsheet or a SQL table, but with a few key differences:

- A DataFrame can have a name.
- A DataFrame can have a data type for each column.
- A DataFrame can have a index, which is a sequence of labels that can be used to access the data.

[1]:
```
# Checking Pandas Version
import pandas as pd
print(pd.__version__)
```

```
3.0.0
```

7.3 Labels ?

If nothing else is specified, the values are labeled using their index number. First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

7.4 Example:

```
print(myvar[0])
```

7.5 Create labels ?

With the index argument, you can name your own labels.

[3]:
```
# Create a simple Pandas Series from a list:
import pandas as pd
a = [1, 7, 2, 9]
myvar = pd.Series(a)
print(myvar)
print(myvar[0])
myvar[0] = 3
print(myvar)
```

```
0    1
1    7
2    2
3    9
dtype: int64
1
0    3
1    7
2    2
3    9
dtype: int64
```

[4]:
```
# Create a simple Pandas Series from a list:
# With the index argument, you can name your own labels.
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a, index = ["x", "y", "z"])
print(myvar)
print("\n ========================================\n ")
#When you have created labels, you can access an item by referring to the label.
print(myvar['y'])
```

```
x    1
y    7
z    2
dtype: int64
```

```
 ========================================
```

```
7
```

[5]:
```
# Create a simple Pandas Series from a dictionary:
# Note: The keys of the dictionary become the labels.
import pandas as pd
calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series(calories, dtype="Int32")
print(myvar)
# To select only some of the items in the dictionary,
# use the index argument and specify only the items you want to include in the␣
 ↪Series.
print("\n ========================================\n ")
myvar2 = pd.Series(calories, index = ["day1", "day2"])
print(myvar2)
print("\n ========================================\n ")
```

```
day1    420
day2    380
day3    390
dtype: Int32

 ========================================

day1    420
day2    380
dtype: int64

 ========================================
```

[6]:
```
# Data sets in Pandas are usually multi-dimensional tables, called DataFrames.
# Series is like a column, a DataFrame is the whole table.
# Create a DataFrame from two Series:
import pandas as pd
data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}
myvar = pd.DataFrame(data, dtype="Int64")
print(myvar)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

[21]:
```
import pandas as pd
#Create a simple Pandas dataframe:
data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}
#load data into a DataFrame object:
df = pd.DataFrame(data)
print(df)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

[8]:
```
import pandas as pd
data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}
df = pd.DataFrame(data)
print(df)
print("\n ========================================\n ")

# Pandas use the loc attribute to return one or more specified row(s)
# refer to the row index:
print(df.loc[1:2])   # Note: This example returns a Pandas Series.
print("\n ========================================\n ")
print (df.calories)
print(type(df.calories))
```

```
   calories  duration
0       420        50
1       380        40
2       390        45

 ========================================

   calories  duration
1       380        40
2       390        45

 ========================================

0    420
1    380
2    390
Name: calories, dtype: int64
```

```
<class 'pandas.Series'>
```

[9]:
```
# Named Indexes
# With the index argument, you can name your own indexes.
# Add a list of names to give each row a name:
from operator import index

import pandas as pd
data = {
  "calories": [420.9, 380.7, 390.4],
  "duration": [50, 40, 45]
}

# load data into a DataFrame object, and begin indexies from 1:
my_data_Frame = pd.DataFrame(data, index= [1, 2, 3])
print(my_data_Frame)
my_data_Frame = pd.DataFrame(data, index = ["d1", "d2", "d3"])

print(my_data_Frame)

print("\n ========================================\n ")
#refer to the named index:
print(my_data_Frame.loc["d3"])
print(my_data_Frame.calories["d1":"d2"])
```

```
   calories  duration
1     420.9        50
2     380.7        40
3     390.4        45
    calories  duration
d1     420.9        50
d2     380.7        40
d3     390.4        45

 ========================================

calories    390.4
duration     45.0
Name: d3, dtype: float64
d1    420.9
d2    380.7
Name: calories, dtype: float64
```

8 Read CSV Files

A simple way to store big data sets is to use CSV files (comma separated files).

CSV files contains plain text and is a well know format that can be read by everyone including

Pandas.

Example:

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.to_string())
```

[ ]:
```
# Load a comma separated file (CSV file) into a DataFrame:
import pandas as pd
df = pd.read_csv('../data/data.csv')
print(df)
print(df.to_string())
```

```
     Duration  Pulse  Maxpulse  Calories
0          60    110       130     409.1
1          60    117       145     479.0
2          60    103       135     340.0
3          45    109       175     282.4
4          45    117       148     406.0
5          60    102       127     300.0
6          60    110       136     374.0
7          45    104       134     253.3
8          30    109       133     195.1
9          60     98       124     269.0
10         60    103       147     329.3
11         60    100       120     250.7
12         60    106       128     345.3
13         60    104       132     379.3
14         60     98       123     275.0
15         60     98       120     215.2
16         60    100       120     300.0
17         45     90       112       NaN
18         60    103       123     323.0
19         45     97       125     243.0
20         60    108       131     364.2
21         45    100       119     282.0
22         60    130       101     300.0
23         45    105       132     246.0
24         60    102       126     334.5
25         60    100       120     250.0
26         60     92       118     241.0
27         60    103       132       NaN
28         60    100       132     280.0
29         60    102       129     380.3
30         60     92       115     243.0
31         45     90       112     180.1
32         60    101       124     299.0
33         60     93       113     223.0
```

34	60	107	136	361.0
35	60	114	140	415.0
36	60	102	127	300.0
37	60	100	120	300.0
38	60	100	120	300.0
39	45	104	129	266.0
40	45	90	112	180.1
41	60	98	126	286.0
42	60	100	122	329.4
43	60	111	138	400.0
44	60	111	131	397.0
45	60	99	119	273.0
46	60	109	153	387.6
47	45	111	136	300.0
48	45	108	129	298.0
49	60	111	139	397.6
50	60	107	136	380.2
51	80	123	146	643.1
52	60	106	130	263.0
53	60	118	151	486.0
54	30	136	175	238.0
55	60	121	146	450.7
56	60	118	121	413.0
57	45	115	144	305.0
58	20	153	172	226.4
59	45	123	152	321.0
60	210	108	160	1376.0
61	160	110	137	1034.4
62	160	109	135	853.0
63	45	118	141	341.0
64	20	110	130	131.4
65	180	90	130	800.4
66	150	105	135	873.4
67	150	107	130	816.0
68	20	106	136	110.4
69	300	108	143	1500.2
70	150	97	129	1115.0
71	60	109	153	387.6
72	90	100	127	700.0
73	150	97	127	953.2
74	45	114	146	304.0
75	90	98	125	563.2
76	45	105	134	251.0
77	45	110	141	300.0
78	120	100	130	500.4
79	270	100	131	1729.0
80	30	159	182	319.2
81	45	149	169	344.0

82	30	103	139	151.1
83	120	100	130	500.0
84	45	100	120	225.3
85	30	151	170	300.0
86	45	102	136	234.0
87	120	100	157	1000.1
88	45	129	103	242.0
89	20	83	107	50.3
90	180	101	127	600.1
91	45	107	137	NaN
92	30	90	107	105.3
93	15	80	100	50.5
94	20	150	171	127.4
95	20	151	168	229.4
96	30	95	128	128.2
97	25	152	168	244.2
98	30	109	131	188.2
99	90	93	124	604.1
100	20	95	112	77.7
101	90	90	110	500.0
102	90	90	100	500.0
103	90	90	100	500.4
104	30	92	108	92.7
105	30	93	128	124.0
106	180	90	120	800.3
107	30	90	120	86.2
108	90	90	120	500.3
109	210	137	184	1860.4
110	60	102	124	325.2
111	45	107	124	275.0
112	15	124	139	124.2
113	45	100	120	225.3
114	60	108	131	367.6
115	60	108	151	351.7
116	60	116	141	443.0
117	60	97	122	277.4
118	60	105	125	NaN
119	60	103	124	332.7
120	30	112	137	193.9
121	45	100	120	100.7
122	60	119	169	336.7
123	60	107	127	344.9
124	60	111	151	368.5
125	60	98	122	271.0
126	60	97	124	275.3
127	60	109	127	382.0
128	90	99	125	466.4
129	60	114	151	384.0

130	60	104	134	342.5
131	60	107	138	357.5
132	60	103	133	335.0
133	60	106	132	327.5
134	60	103	136	339.0
135	20	136	156	189.0
136	45	117	143	317.7
137	45	115	137	318.0
138	45	113	138	308.0
139	20	141	162	222.4
140	60	108	135	390.0
141	60	97	127	NaN
142	45	100	120	250.4
143	45	122	149	335.4
144	60	136	170	470.2
145	45	106	126	270.8
146	60	107	136	400.0
147	60	112	146	361.9
148	30	103	127	185.0
149	60	110	150	409.4
150	60	106	134	343.0
151	60	109	129	353.2
152	60	109	138	374.0
153	30	150	167	275.8
154	60	105	128	328.0
155	60	111	151	368.5
156	60	97	131	270.4
157	60	100	120	270.4
158	60	114	150	382.8
159	30	80	120	240.9
160	30	85	120	250.4
161	45	90	130	260.4
162	45	95	130	270.0
163	45	100	140	280.9
164	60	105	140	290.8
165	60	110	145	300.0
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

[1]:
```
# pandas series from yfinance using apple Close data then save to AAPL.csv file␣
↪and load it back to a dataframe and print it
import yfinance as yf
import pandas as pd

# Get the data for the stock AAPL
data = yf.download('AAPL', start='2025-01-01', end='2025-12-31')
```

```
data.to_csv('data/AAPL.csv')
# Load a comma separated file (CSV file) into a DataFrame:
df = pd.read_csv('data/AAPL.csv')
print(df)
# Pandas - Display Options
# Pandas has some display options that can be changed.

# max_columns
# The number of columns returned is defined in Pandas option settings.
```

```
[**********************100%**********************]  1 of 1 completed
          Price                Close                 High                  Low  \
0        Ticker                 AAPL                 AAPL                 AAPL
1          Date                  NaN                  NaN                  NaN
2    2025-01-02  242.52516174316406  247.74663833648884  240.50619200781614
3    2025-01-03  242.03781127929688    242.853348279154  240.57579666249651
4    2025-01-06   243.6688995361328  245.98624232017474   241.8786760145675
..          …                   …                   …                   …
246  2025-12-23   272.1053771972656   272.2452609569047  269.30800689497966
247  2025-12-24  273.55401611328125  275.17249671786806   271.9455359163183
248  2025-12-26   273.1444091796875   275.1125687682936      272.6049054553
249  2025-12-29  273.50408935546875  274.10350406329195   272.0954038139857
250  2025-12-30     272.82470703125  273.82377221524894  272.02546707967076

                   Open    Volume
0                  AAPL      AAPL
1                   NaN       NaN
2    247.57754859678104  55740700
3    242.03781127929688  40244100
4     242.9826459215422  45045600
..                  …       …
246   270.5868091030317  29642000
247    272.085389177814  17910600
248   273.9037084592662  21521800
249   272.4350823114352  23715200
250   272.5549704076282  22139600

[251 rows x 6 columns]
```

[ ]:
```
# max_rows
# The number of rows returned is defined in Pandas option settings.
# You can check your system's maximum rows with the pd.options.display.max_rows␣
 ↪statement.
```

```
import pandas as pd
print(pd.options.display.max_rows)
# You can change the maximum rows number with the same statement.
pd.options.display.max_rows = 9999
df = pd.read_csv('../data/data.csv')
print(df)
```

[ ]:
```
import pandas as pd
df = pd.read_csv('../data/data.csv')
print(df)
print("\n ========================================\n")
print(df)

print(pd.options.display.max_rows)
```

9 Read JSON Files

Big data sets are often stored, or extracted as JSON.

JSON is plain text, but has the format of an object, and is well known in the world of programming, including Pandas.

Example:

```
import pandas as pd
df = pd.read_json('data.json')
print(df.to_string())
```

[17]:
```
import pandas as pd
df = pd.read_json('../data/data.json')
print(df)
```

```
     Duration  Pulse  Maxpulse  Calories
0          60    110       130     409.1
1          60    117       145     479.0
2          60    103       135     340.0
3          45    109       175     282.4
4          45    117       148     406.0
..        ...    ...       ...       ...
164        60    105       140     290.8
165        60    110       145     300.4
166        60    115       145     310.2
167        75    120       150     320.4
168        75    125       150     330.4

[169 rows x 4 columns]
```

[18]:
```
# JSON = Python Dictionary
# JSON objects have the same format as Python dictionaries.
```

```
# If your JSON code is not in a file, but in a Python Dictionary,
# you can load it into a DataFrame directly:

# Load a Python Dictionary into a DataFrame:

import pandas as pd
data = {
  "Duration":{
    "0":60,
    "1":60,
    "2":60,
    "3":45,
    "4":45,
    "5":60
  },
  "Pulse":{
    "0":110,
    "1":117,
    "2":103,
    "3":109,
    "4":117,
    "5":102
  },
  "Maxpulse":{
    "0":130,
    "1":145,
    "2":135,
    "3":175,
    "4":148,
    "5":127
  },
  "Calories":{
    "0":409,
    "1":479,
    "2":340,
    "3":282,
    "4":406,
    "5":300
  }
}
df = pd.DataFrame(data)
print(df)
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130       409
1        60    117       145       479
2        60    103       135       340
3        45    109       175       282
```

```
4        45     117        148        406
5        60     102        127        300
```

10 Pandas - Analyzing DataFrames

10.1 Viewing the Data

One of the most used method for getting a quick overview of the DataFrame, is the head() method. The head() method returns the headers and a specified number of rows, starting from the top.

[13]:
```
# Get a quick overview by printing the first 10 rows of the DataFrame:
import pandas as pd
df = pd.read_csv('../data/data.csv')
print(df.head())

# Note: if the number of rows is not specified, the head() method will return␣
 ↪the top 5 rows.
print("\n ========================================\n")
df2 = pd.read_csv('../data/data.csv')
print(df2.head(6))
```

```
   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0

 ========================================

   Duration  Pulse  Maxpulse  Calories
0        60    110       130     409.1
1        60    117       145     479.0
2        60    103       135     340.0
3        45    109       175     282.4
4        45    117       148     406.0
5        60    102       127     300.0
```

There is also a tail() method for viewing the last rows of the DataFrame. The tail() method returns the headers and a specified number of rows, starting from the bottom.

[24]:
```
# There is also a tail() method for viewing the last rows of the DataFrame.
# The tail() method returns the headers and a specified number of rows,␣
 ↪starting from the bottom.
import pandas as pd
df = pd.read_csv('../data/data.csv')
print(df.tail(7))
```

```
     Duration  Pulse  Maxpulse  Calories
```

```
162        45     95        130     270.0
163        45    100        140     280.9
164        60    105        140     290.8
165        60    110        145     300.0
166        60    115        145     310.2
167        75    120        150     320.4
168        75    125        150     330.4
```

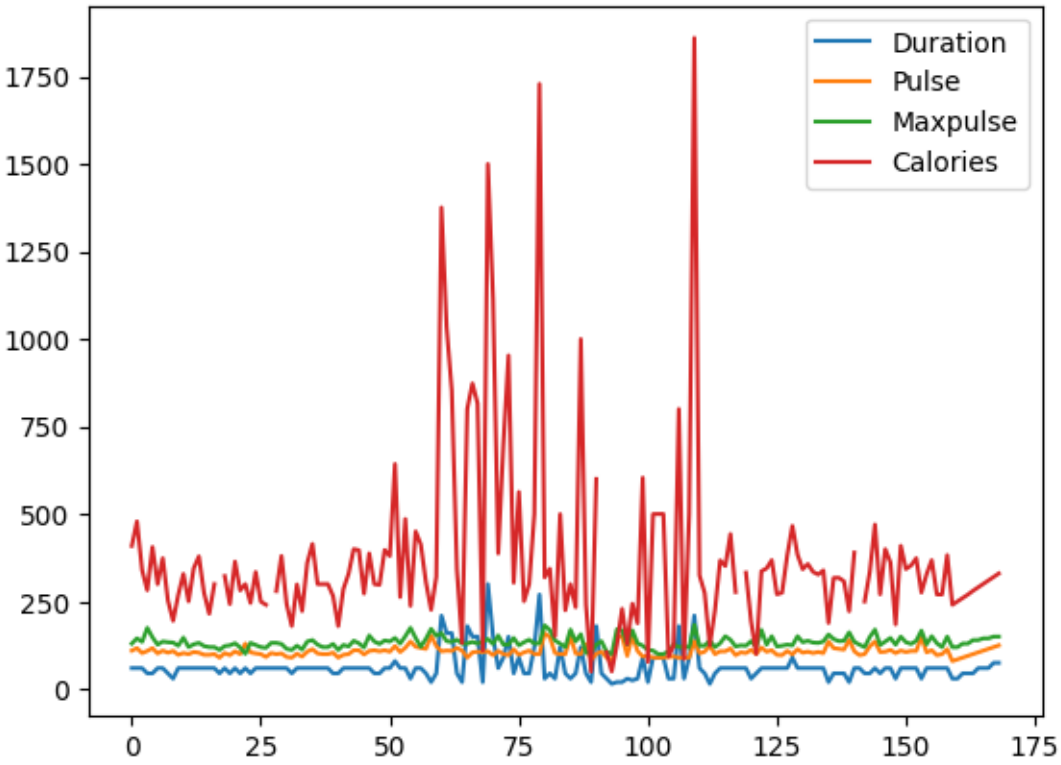
10.2 Info About the Data

The DataFrames object has a method called info(), that gives you more information about the data set.

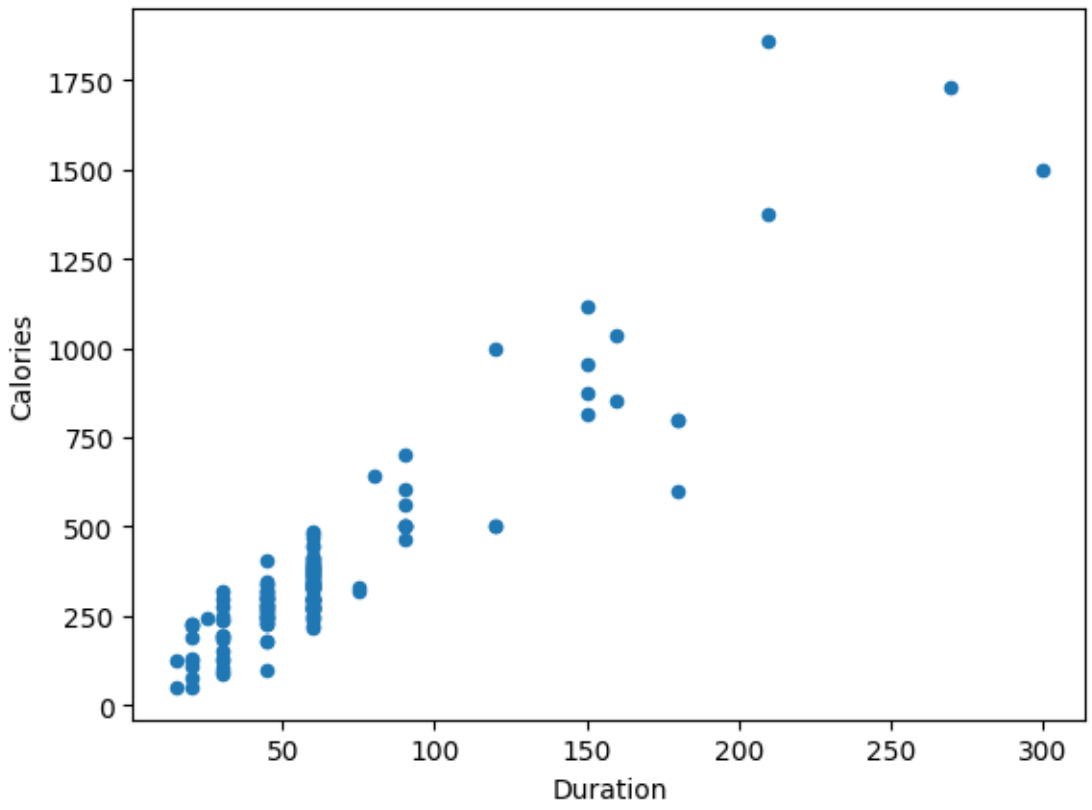
[25]:
```
import pandas as pd
df = pd.read_csv('../data/data.csv')
print(df.info())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 4 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Duration  169 non-null    int64
 1   Pulse     169 non-null    int64
 2   Maxpulse  169 non-null    int64
 3   Calories  164 non-null    float64
dtypes: float64(1), int64(3)
memory usage: 5.4 KB
None
```

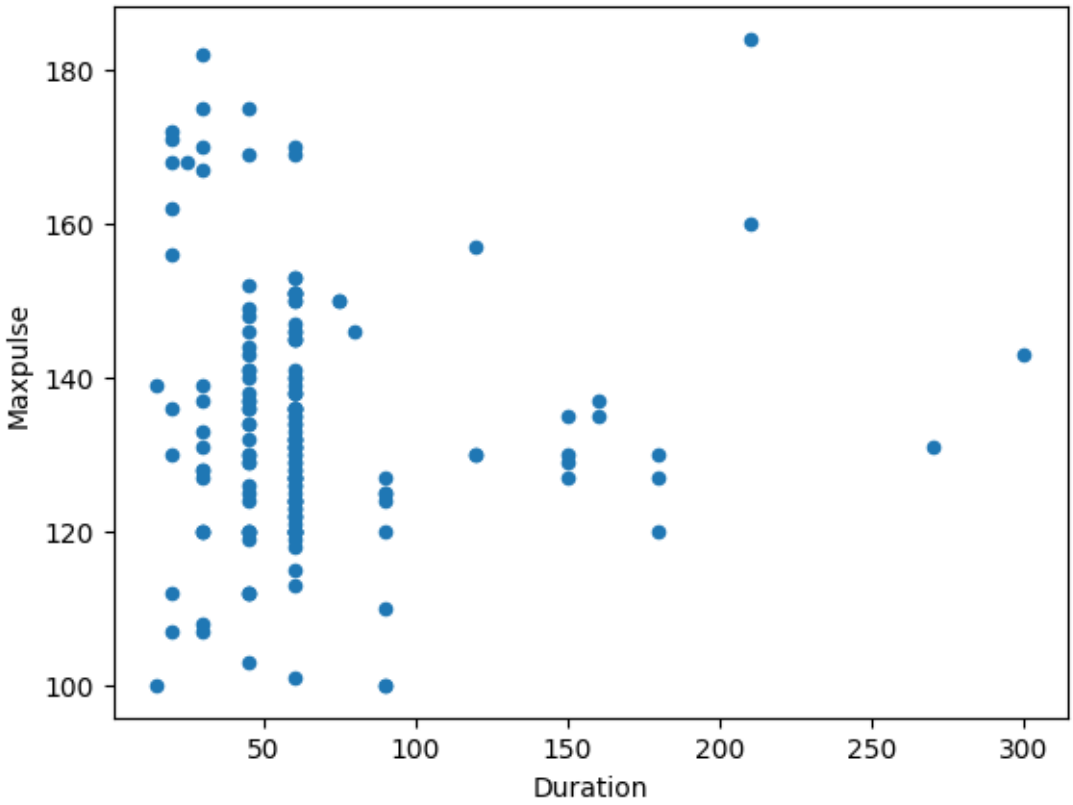
[ ]:
```
# Import pyplot from Matplotlib and visualize our DataFrame:
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('../data/data.csv')
df.plot()
plt.show()
```



[27]:
```
df.plot(kind ='scatter', x = 'Duration', y = 'Calories')
plt.show()
```

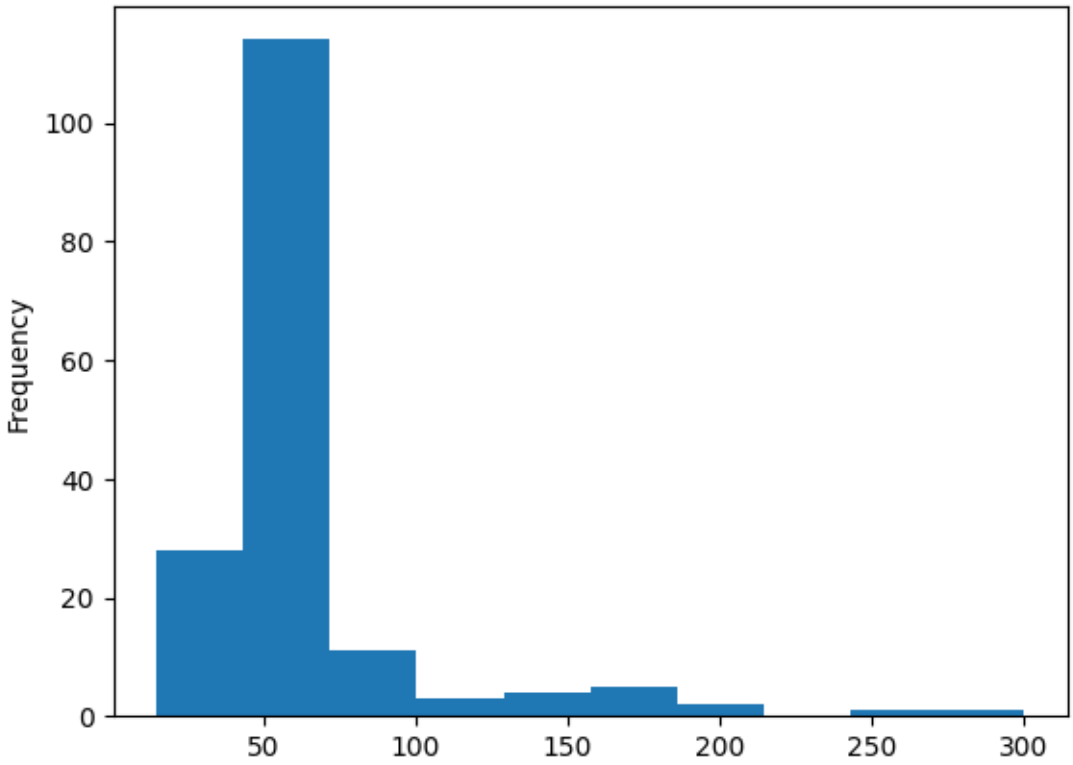


[28]:
```
df.plot(kind = 'scatter', x = 'Duration', y = 'Maxpulse')
plt.show()
```



[29]:
```
df["Duration"].plot(kind = 'hist')
```

[29]: <Axes: ylabel='Frequency'>



11 Pandas - Data Correlations

11.1 1. Finding Relationships

A great aspect of the Pandas module is the corr() method. The corr() method calculates the relationship between each column in your data set. The examples in this page uses a CSV file called: 'data.csv'.